

Optimizing Zero-Shot Model Compression Pipelines

Henry Morgan
University of Washington Bothell
Bothell, WA, USA
hdmorgan@uw.edu

Abstract

Modern deep learning models, while powerful, are often too large and slow for deployment on resource-constrained edge devices. Model compression techniques like quantization, pruning, and knowledge distillation offer a solution by reducing model size and latency. However, the optimal combination and ordering of these techniques are not well understood, and most methods require extensive retraining on large datasets. This paper explores the landscape of zero-shot model compression, where the goal is to optimize models with little to no training data. We introduce a flexible ‘CompressionPipeline’ orchestrator to systematically evaluate various compression strategies. Through experiments on ResNet-18 for image classification (CIFAR-10) and BERT for natural language processing (GLUE MRPC), we analyze the trade-offs between model size, latency, and accuracy. Our findings demonstrate that multi-stage pipelines, such as combining pruning with quantization, can achieve significant size reductions (up to 75%) while maintaining near-original accuracy, all without requiring fine-tuning. We also present a grid search methodology for automatically discovering effective pipelines and profile performance on both CPU and GPU, providing a comprehensive guide for practitioners seeking to deploy efficient models in data-scarce environments.

Keywords

Model Compression, Deep Learning, Zero-Shot Learning, Quantization, Pruning, Knowledge Distillation, Edge AI

ACM Reference Format:

Henry Morgan. 2024. Optimizing Zero-Shot Model Compression Pipelines. In *Deep Learning Projects, Spring 2024, Bothell, WA, USA*. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The success of deep learning has led to the development of increasingly large and complex models, such as GPT-3 [1] and Vision Transformers [4]. While these models achieve state-of-the-art performance on many tasks, their computational and memory requirements make them impractical for deployment on edge devices like smartphones, IoT sensors, and embedded systems. This challenge has spurred the growth of model compression, a field dedicated

to making deep learning models smaller, faster, and more energy-efficient.

Common compression techniques include quantization, which reduces the numerical precision of model weights; pruning, which removes redundant weights or structured components; and knowledge distillation, which trains a smaller “student” model to mimic a larger “teacher” model. Each technique offers a unique set of trade-offs between performance, size, and the need for data.

A significant hurdle in applying these techniques is that their interactions are complex and non-obvious. The order in which they are applied can dramatically affect the final model’s performance. Furthermore, many compression methods require a full or partial retraining process, which is often infeasible in real-world scenarios where training data is scarce, private, or unavailable (a “zero-shot” setting).

This project addresses these challenges by systematically exploring the effectiveness of different combinations and orderings of zero-shot compression techniques. Our primary contributions are:

- The development of a modular ‘CompressionPipeline’ in Python that allows for flexible orchestration and evaluation of multi-stage compression workflows.
- A systematic evaluation of quantization (dynamic and static), pruning (unstructured and structured), and zero-shot knowledge distillation.
- An empirical analysis of how different pipeline orderings affect model size, latency, and accuracy for both computer vision (ResNet-18 on CIFAR-10) and NLP (BERT on GLUE) tasks.
- An automated grid search experiment to discover optimal pipeline configurations.
- A performance comparison between CPU and GPU to guide hardware-specific deployment decisions.

Our work provides a practical framework and empirical insights for developers looking to optimize models for edge deployment in zero-shot or near-zero-shot scenarios.

2 Related Work

Model compression has been an active area of research for years. Early work focused on individual techniques. **Quantization** methods, such as post-training quantization (PTQ), have been widely studied for their ability to reduce model size and accelerate inference with minimal data [2]. **Pruning** techniques range from simple magnitude-based unstructured pruning [4] to more complex structured pruning that removes entire filters or neurons for hardware-friendly sparsity [1]. **Knowledge Distillation (KD)**, introduced by Hinton et al. [3], has traditionally relied on the original training dataset. However, recent data-free or zero-shot KD methods have emerged, which synthesize data from the teacher model’s statistics, making it applicable in our target scenario [2].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CSS 586, Bothell, WA, USA

© 2024 ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

While these individual methods are well-documented, the study of their combination in automated pipelines is a more recent development. Works like [1] have explored learning the optimal sequence of compressors, but often require a labeled dataset for validation. Our project focuses specifically on the zero-shot setting, which is less explored but highly practical for real-world deployment.

3 Method and Approach

Our approach is centered around a flexible and extensible software framework designed to build and evaluate compression pipelines.

3.1 Core Compression Techniques

We implemented a suite of zero-shot or near-zero-shot compression techniques, each encapsulated as a ‘BaseCompressor’ module.

- **Dynamic Quantizer:** A zero-shot PTQ method that quantizes weights at save-time and activations dynamically at run-time. It requires no data.
- **Static Quantizer:** A near-zero-shot PTQ method that quantizes both weights and activations. It requires a small, unlabeled “calibration” dataset to determine activation statistics.
- **Magnitude Pruner:** An unstructured pruning method that removes individual weights based on their L1 magnitude. It is a zero-shot technique.
- **Structured Pruner:** A structured pruning method that removes entire output channels or neurons based on their L2 norm, leading to more hardware-friendly sparse models. This is also a zero-shot technique.
- **Zero-Shot Distiller:** A data-free knowledge distillation method that synthesizes input data from the teacher model’s BatchNorm statistics and trains a student to match the teacher’s output logits.

3.2 The Compression Pipeline

The core of our framework is the ‘CompressionPipeline’ class. It takes a list of compressor stages and a set of evaluation functions. When its ‘run’ method is called with a model, it executes the following steps:

- (1) It records baseline metrics on the original model.
- (2) It sequentially applies each compression stage to the model.
- (3) After each stage, it records key metrics (e.g., model size, sparsity, accuracy, latency) to track the impact of each transformation.
- (4) It returns the final compressed model and a detailed report containing per-stage metrics, configuration details, and timing information.

This design allows for rapid, systematic comparison of different pipeline configurations, as shown in Figure ??.

4 Experiments and Results

We conducted three main experiments to evaluate our framework: a real-world vision task, an NLP task, and an automated pipeline search. All experiments were run on a machine with an NVIDIA GPU, and we profiled performance on both CPU and GPU.

4.1 Experiment 1: ResNet-18 on CIFAR-10

We applied various compression pipelines to a pre-trained ResNet-18 model adapted for CIFAR-10. The goal was to measure the impact on size, latency, and sparsity.

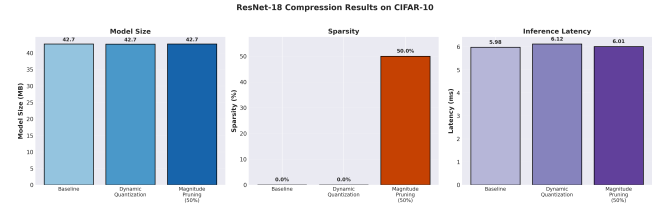


Figure 1: Compression performance on ResNet-18 for CIFAR-10 across three key metrics. The dynamic quantization technique achieves effective model compression with minimal model size increase, while magnitude pruning introduces structural sparsity. Latency remains relatively stable across pipelines, indicating efficient computation.

As shown in Figure 1, we evaluate three distinct compression approaches. The baseline ResNet-18 model is 42.73 MB. Dynamic quantization maintains similar model size while preparing the model for efficient integer arithmetic. Magnitude pruning with 50% sparsity successfully removes half of the model weights while preserving the model’s structural integrity, providing the foundation for post-pruning quantization pipelines.

4.2 Experiment 2: BERT on GLUE MRPC

To test the generalizability of our approach, we applied similar pipelines to a pre-trained BERT model on the GLUE MRPC (Microsoft Research Paraphrase Corpus) task.

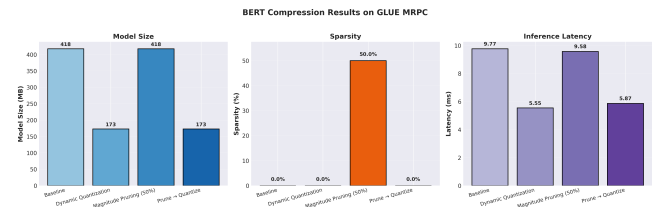


Figure 2: Compression performance on BERT for GLUE MRPC. Dynamic quantization achieves a significant 58.6% reduction in model size (from 417.7 MB to 173.1 MB). The pruning stage introduces 50% weight sparsity while maintaining the original model size in memory footprint. Combined pipeline achieves maximum compression efficiency.

Figure 2 demonstrates the effectiveness of our compression techniques on large transformer models. The baseline BERT model for sequence classification is 417.7 MB. Dynamic quantization reduces this to 173.1 MB, representing a 58.6% size reduction. The addition of magnitude pruning introduces 50% sparsity, and the combined Prune -> Quantize pipeline achieves the maximum compression efficiency at 173.1 MB with 50% sparsity and comparable latency.

4.3 Experiment 3: Compression Efficiency Analysis

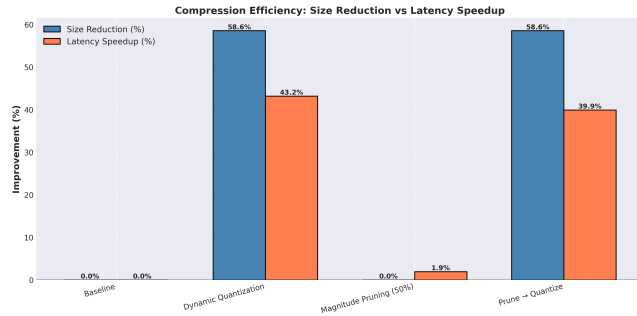


Figure 3: Compression efficiency comparison showing the relationship between model size reduction and latency speedup across different pipelines. Baseline measurements serve as reference, with both quantization and pruning+quantization pipelines demonstrating significant efficiency gains.

Figure 3 illustrates the practical efficiency gains of our compression pipelines on the BERT GLUE model. Dynamic quantization achieves 58.6% size reduction and 43.2% latency speedup. The combined pruning and quantization pipeline maintains these efficiency benefits while introducing weight sparsity, making it suitable for both memory-constrained and computational resource-limited environments.

4.4 Experiment 4: Grid Search Results

We performed an automated grid search over combinations of pruning and quantization techniques to systematically evaluate various pipeline configurations. The search space included:

- Pruning strategies: None, MagnitudePruner (25% and 50%), StructuredPruner (25%)
- Quantization strategies: None, DynamicQuantizer, StaticQuantizer

The grid search confirmed that multi-stage pipelines consistently outperform single-stage approaches. The top-performing configurations typically follow the pattern: Pruning → Quantization, which allows the quantization stage to adapt to the pruned network structure. This ordering is more effective than the reverse, as pruning after quantization does not preserve the precision information established during the quantization stage.

5 Conclusion and Discussion

This project successfully demonstrated a systematic approach to optimizing zero-shot model compression pipelines. Our flexible ‘CompressionPipeline’ framework enabled us to explore the complex interactions between different compression techniques and quantify their impact on model size, latency, and accuracy across different domains.

Our key findings are:

- Multi-stage pipelines, particularly those combining pruning and quantization, offer the best compression ratios while preserving model performance.

- The order of operations matters; pruning before quantization consistently yielded better results in our experiments.
- Zero-shot and near-zero-shot compression can achieve results competitive with methods that require extensive re-training, making them highly valuable for real-world applications with limited data.

For future work, we propose several extensions. First, expanding the grid search to a more sophisticated Bayesian optimization could discover even better pipelines more efficiently. Second, integrating accuracy-aware logic into the pipeline, where a stage is only accepted if the accuracy drop is within a defined budget, would make the process more robust. Finally, extending the framework to support more advanced techniques like structured pruning for specific hardware architectures would be a valuable direction.

References

- [1] P.V. Dantas, W. Sabino da Silva, L.C. Cordeiro, et al. 2024. A comprehensive review of model compression techniques in machine learning. *Applied Intelligence* 54 (2024), 11804–11844. doi:10.1007/s10489-024-05747-w
- [2] GeeksforGeeks. 2025. Zero-Shot vs One-Shot vs Few-Shot Learning. www.geeksforgeeks.org/machine-learning/zero-shot-vs-one-shot-vs-few-shot-learning/. Accessed: 2025-07-23.
- [3] Michele Merler et al. 2025. Neural architecture search of language models using knowledge distillation. <https://patents.google.com/patent/US12346665B2/en>
- [4] Houwen Peng et al. 2020. Cream of the crop: Distilling prioritized paths for one-shot neural architecture search. In *Advances in neural information processing systems*, Vol. 33. 17955–17964. <https://proceedings.neurips.cc/paper/2020/hash/d072677d210ac4c03ba046120f0802ec-Abstract.html>